Transformers: Interview Mind Map

Oct 20, 2026 · @Mukundan

Overview

A Transformer is a neural network built for sequences that looks at the whole sequence at once instead of stepping through it one element at a time. It came from the 2017 paper "Attention Is All You Need" and is the architecture behind modern large language models.

The key break from RNNs and LSTMs: no recurrence. A Transformer uses a mechanism called self-attention to let every element look directly at every other element, in parallel. That removes the forgetting problem and makes training far faster on modern hardware.

One line to anchor the map: an RNN reads a sentence word by word, a Transformer reads the whole sentence at once and decides what matters.

Intuition in plain words

Picture a group of people in a room, each holding one word of a sentence. To understand their own word, each person is allowed to look around and ask everyone else, "how relevant are you to me?" They pay more attention to the words that matter and ignore the rest. Everyone does this at the same time, so the whole sentence is understood in one pass.

That is self-attention. In "the animal did not cross the street because it was tired," the word "it" looks around, decides "animal" matters most, and links to it. No stepping through word by word, no running summary to forget.

An LSTM reads with a notebook so it does not forget. A Transformer does not need a notebook, because every word can see every other word directly, all at once.

Self-attention: the core trick

Every word is turned into three vectors, and the names are worth memorizing:

1. Query (Q): what this word is looking for.
2. Key (K): what this word offers, like a label.
3. Value (V): the actual content this word carries.

Attention works in three steps. Each word's Query is compared against every word's Key to get a relevance score. Those scores are softmaxed into weights that sum to 1. Each word then takes a weighted blend of all the Values, so it pulls in the most from the words that matter.

The standard formula, good to recognize:

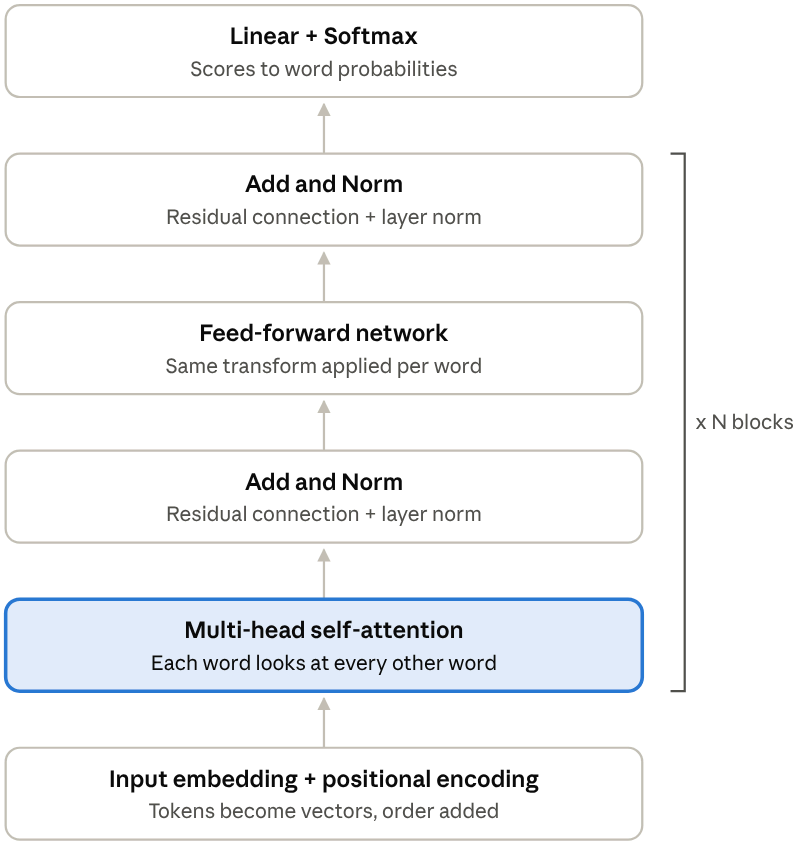
```
\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q K^{\top}}
{\sqrt{d_k}}\right) V
```

The division by the square root of d_k just keeps the scores from getting too large before the softmax.

Multi-head attention runs this several times in parallel, each "head" learning a different kind of relationship (one tracks grammar, another tracks meaning, and so on). The heads are then combined. More heads means more types of relationships captured at once.

Architecture

Transformer: every word attends to every other, in parallel



Transformer block · input embedding to output probabilities

Read the block bottom to top. Real models stack N of these.

- Input embedding + positional encoding: each token becomes a vector. Since attention has no sense of order, a positional encoding is added so the model can tell word 1 from word 5.
- Multi-head self-attention: the core step. Every word builds a Query, Key and Value, then pulls in information from the words most relevant to it. Several heads capture different relationships at once.
- Add & Norm: a residual connection adds the layer's input back to its output, then layer normalization keeps the numbers stable. This is what lets very deep stacks train well.
- Feed-forward network: a small network applied to each position on its own, giving the model room to transform what attention gathered.
- Add & Norm: the same residual plus normalization after the feed-forward layer.

- Linear + Softmax: the output head turns the final vectors into probabilities over the vocabulary.

Encoder vs decoder: an encoder block is exactly this. A decoder block adds two things, a masked self-attention so it cannot peek at future words, and a cross-attention layer that reads the encoder's output. Encoder-only models (BERT) understand text, decoder-only models (GPT) generate it.

Why it beat RNNs and LSTMs

- Parallelism: an RNN must process word 1 before word 2. A Transformer processes all words at once, so it trains far faster on GPUs.
- Long-range context: attention links any two words directly, no matter how far apart. An LSTM has to carry that information step by step and can still lose it.
- Scales with data and size: stacking more layers and training on more text keeps improving results, which is what made today's large language models possible.

The trade-off: attention compares every word with every other word, so cost grows with the square of the sequence length. Long inputs get expensive, which is an active research area.

Model	Reads sequence	Long-range memory	Training speed
RNN	One step at a time	Weak	Slow
LSTM	One step at a time	Better, via gates	Slow
Transformer	All at once	Strong, via attention	Fast (parallel)

Common interview follow-ups

- Why do Transformers need positional encoding? Attention alone has no sense of word order, so position has to be added into the embeddings.
- What are Q, K and V? Query is what a word is looking for, Key is what each word offers, Value is the content that gets blended in by the attention weights.
- Why multiple heads? Each head can learn a different relationship (syntax, meaning, reference), and combining them captures more at once.

- Why divide by the square root of d_k? To stop the dot products from growing too large, which would make the softmax too sharp and hurt gradients.
- What is the main weakness? Attention cost grows with the square of sequence length, so very long inputs are expensive.
- Encoder-only vs decoder-only? BERT is encoder-only and is built to understand text. GPT is decoder-only and is built to generate it.